

ScholarFlow — Demo Video Speech

Team Phantom Devs · Total duration: **12:15** (Introduction 1:40 · Core Features Demo 8:45 · Testing Demo 1:15 · GitHub & Close 0:35)

Introduction (0:00 – 1:40)

Opening (0:00 – 0:20)

Every researcher knows this feeling. Papers live in one app. Notes in another. Citations somewhere else. And the AI knows nothing about your actual work. This is the problem ScholarFlow solves.

What ScholarFlow Is (0:20 – 0:45)

Our team is Phantom Devs, and ScholarFlow is one place for the full research workflow. Upload a paper, understand it with AI, annotate it, write with it, cite it, and collaborate on it in real time. No existing tool combines all of these. ScholarFlow is the first.

Built to Be Trusted (0:45 – 1:25)

And ScholarFlow is more than a paper library. It is the reading tool, the writing tool, the citation manager, the AI assistant, and the shared workspace — working together in one product. And because research must be trusted, correctness, permissions, and payment safety are engineered in from the start.

Setup (1:25 – 1:40)

I am logged in as Bob, a team lead. My library already has papers, and my team is active. Let me start with the most common task — adding a new paper.

Core Features Demo (1:40 – 10:25)

1. Upload and Automatic Metadata (1:40 – 2:30)

This is the upload page, and the paper is ready on my desktop. I drop it here — and while it uploads, ScholarFlow reads the document in the background. Watch the metadata panel:

the title, the authors, and the abstract appear on their own, extracted by AI from the PDF. I change nothing. I click publish, and the paper joins my library in secure cloud storage.

Let me find it the way a researcher would — by meaning.

2. Library and Semantic Search (2:30 – 3:10)

This is my library — papers, tags, status, and collections. But now I do not search by exact words; I search by meaning. I type "language models hallucinate", and the system finds the relevant paper, even though those words are not in its title. That is semantic search, powered by vector embeddings — and it only searches papers I am allowed to see.

3. Collections (3:10 – 3:35)

Search finds the paper; collections keep it organized. I add it to a collection — or create a new one, like "Thesis Reading". And collections carry permissions, so I can share a whole collection with my team. Every paper keeps its own tags and status, so a collection stays a clean, curated list.

4. Discover and Import (3:35 – 4:15)

And where do new papers come from? Two ways. First, import: I paste a DOI or an arXiv link, and the paper and its metadata are fetched automatically. Second, discovery: this is the Discover feed — live trending papers and recommendations based on my reading. One click saves it to my library. And the import runs the same AI metadata extraction, so a new paper arrives ready to read.

5. Reading, Highlighting, and Notes (4:15 – 4:55)

Found it. Now let me read it. The extracted text is ready, and the PDF preview is right here. I select a sentence, highlight it, and add a short note. I can also keep my own research notes beside the paper. Everything I mark is saved instantly and stays attached — for me, and for my team.

6. AI Summary, Key Points, and Questions (4:55 – 5:55)

Reading is only half the work; the hard part is understanding. So let me open the AI layer. I click generate summary. In a few seconds I get the key findings, in the tone and length I chose. I can also regenerate it with a different tone — for a general audience, or a technical one. Below it are the key points — the main claims of the paper, extracted one by one. And I can ask questions, like: which method did they use? The answer comes from this paper's

own text, not from the open internet. That is the difference between a chatbot and a research assistant.

7. AI Compare and Literature Review (5:55 – 6:30)

And the AI works across papers, not just inside one. I select two papers and compare them — the model shows where they agree and where they disagree. Over a whole collection, I can generate a literature review draft — a synthesis of many papers in seconds. This is the part that saves days of reading: instead of opening fifty tabs, I get one structured overview — and I can still open every source behind it.

8. Writing: Template, Citation, Export (6:30 – 7:30)

Understanding leads to writing — so let me write with it. I create a new paper from a template; this one is IEEE. The editor supports tables, images, and LaTeX math, and it saves automatically — watch the save indicator. I never pressed save. I insert a citation from my library, in the format I choose — one of nine formats, including APA, IEEE, and BibTeX. And when the draft is ready, one click exports a clean PDF. Writing, references, and export — in one place.

9. Real-Time Co-Editing (7:30 – 8:35)

A paper is rarely written alone — so let me show you the part I am most proud of. On the left is my screen. On the right, my teammate Emily, in another browser. Watch: when she types, the text appears on my screen instantly — no refresh, no conflicts. Her cursor carries her name; mine carries mine, so we always know who is where. Two people, one paper, at the same time — real-time co-editing, built into the research workflow.

10. Team Invite and Live Notification (8:35 – 9:10)

And the same live system runs everything around the paper. For example, I invite a new member and choose a role — Viewer, Editor, or Manager. The moment I send it, the notification arrives in the bell, in real time. Invitations, role changes, and removals all flow through this same channel.

11. Payments and Billing (9:10 – 9:55)

Features like these need a business model, so here is the last piece — payments. I upgrade to the Pro plan. This is Stripe Checkout, hosted by Stripe, so no card data touches our servers. I use a test card, and the payment succeeds. The billing page shows my plan and

my invoices. And in the admin console, the same payment appears instantly, because our backend processes Stripe webhooks.

12. Admin Console (9:55 – 10:25)

And behind the product is the operator view. This is the admin console: real-time system metrics — CPU, memory, storage, and database. Reports with exports, the audit log, and platform settings. And every action is recorded, so nothing happens silently. Everything an operator needs, on real data.

Testing Demo (10:25 – 11:40)

Why Testing Matters (10:25 – 10:30)

That is the product. But how do we know it works? We test it.

1. Automated Tests — Jest and Supertest (10:30 – 11:10)

We test it in two ways. These are the automated tests — Jest for unit and integration, Supertest for API end-to-end. They already cover billing, payments, papers, validation, and access control. Watch the suite run — twenty-one suites, fifty-nine tests, all passing.

2. Access Control (11:10 – 11:25)

Next, one live security check: a view-only colleague tries to edit — the server refuses. Four oh three.

3. Payment Safety (11:25 – 11:40)

And money deserves the same care: for payments, webhooks are the source of truth. I replay the same event twice — the subscription changes only once.

GitHub & Engineering Practice (11:40 – 12:15)

Built with Industry Practice (11:40 – 12:05)

Before we finish, one last look — our GitHub — because the process matters too. Sprint-based development, contributions from every member, and every version released

properly. Every feature was reviewed before it merged — the pull requests and the history are all here. And the product is deployed live.

Closing (12:05 – 12:15)

That is ScholarFlow — built by a team and released step by step. Thank you for watching.